

# **Specyfikacja wymagań systemowych**

SRS (Student Planner)

Desktopowa aplikacja produktywnościowa  
Inżynieria Oprogramowania  
7 czerwca 2026

# 1. Wstęp

## 1.1 Cel projektu

Celem projektu jest wdrożenie aplikacji Student Planner, służącej do optymalizacji zarządzania czasem studenta, bieżącego monitorowania postępów w nauce oraz ewidencji kluczowych terminów akademickich. System udostępnia szereg widoków prezentujących informacje w sposób przejrzysty i intuicyjny, umożliwiając efektywną organizację obowiązków.

## 1.2 Konwencja dokumentu

Nie dotyczy.

## 1.3 Adresaci dokumentu oraz wskazówki dotyczące lektury

Niniejsza dokumentacja stanowi merytoryczne kompendium wiedzy na temat systemu Student Planner, dedykowane zarówno użytkownikom końcowym, jak i zespołowi deweloperskiemu. Jej celem jest szczegółowe przedstawienie założeń funkcjonalnych oraz architektury systemu, co pozwala na pełne zrozumienie logiki biznesowej oraz technicznych aspektów wdrożonego rozwiązania.

W treści opracowania zawarto syntetyczne ujęcie celów projektu, kompleksowy opis modułów systemu oraz specyfikację interfejsów użytkownika. Dokumentacja ta służy jako punkt odniesienia w procesie dalszego utrzymania i rozwoju aplikacji. Szczególną uwagę rekomendujemy skierować na rozdziały 2.2 oraz 3.1, w których zawarto kluczowe założenia projektowe oraz definicje funkcjonalności stanowiące fundament całości systemu.

## 1.4 Zakres produktu

System Student Planner stanowi scentralizowane „centrum zarządzania” procesami akademickimi w formie lekkiej i wydajnej aplikacji desktopowej. Aplikacja integruje najważniejsze obszary organizacji życia studenckiego, zapewniając automatyzację w monitorowaniu terminów oraz limitów. Dzięki temu użytkownik zyskuje możliwość koncentracji na procesie dydaktycznym.

Kluczowym założeniem architektonicznym jest zapewnienie niezależności od połączenia sieciowego, co eliminuje ryzyko niedostępności danych w przypadku awarii serwerów uczelnianych. Zastosowanie lokalnej bazy danych SQLite umożliwia pracę w trybie *offline*, gwarantując natychmiastowy dostęp do zasobów. Rozwiązanie to zapewnia wysoki poziom prywatności, gdyż wszystkie dane użytkownika przechowywane są wyłącznie lokalnie.

## 2. Opis ogólny

### 2.1 Perspektywa produktu

Aplikacja Student Planner została opracowana jako autorskie rozwiązanie, powstałe od podstaw w ramach bieżącego procesu twórczego.

### 2.2 Funkcje produktu

- Zarządzanie przedmiotami: deklaracja oraz przechowywanie danych o przedmiotach; modyfikacja parametrów.
- Kalendarz: podgląd harmonogramu zajęć i zdarzeń; obsługa notatek do zajęć.
- Śledzenie postępów: monitorowanie punktów (z aktywności i kolokwiów) wraz z wartościami maksymalnymi oraz ewidencja nieobecności.
- Funkcje czasowe: odliczanie do sesji egzaminacyjnej.
- Konfiguracja: zmiana języka interfejsu (polski/angielski).
- Zarządzanie wydarzeniami: rejestracja, podgląd oraz usuwanie zdarzeń (egzaminy, projekty).
- Powiadomienia: centrum powiadomień informujących o nadchodzących zdarzeniach.

### 2.3 Charakterystyka użytkowników

Grupę docelową stanowią studenci uczelni wyższych, wymagający narzędzia wspierającego organizację semestru. Aplikacja stanowi rozwiązanie alternatywne dla systemów dziekańskich, kładąc nacisk na wysoką ergonomię i czytelność danych.

### 2.4 Środowisko uruchomieniowe

Architektura rozwiązania typu *lightweight* zapewnia niskie wymagania zasobowe, co przekłada się na płynne działanie na zróżnicowanych konfiguracjach sprzętowych. Dzięki skompilowaniu aplikacji do postaci pliku wykonywalnego, końcowy użytkownik nie musi posiadać zainstalowanego środowiska programistycznego ani zewnętrznych bibliotek Kivy w systemie – wszystkie niezbędne komponenty zostały zintegrowane wewnątrz dystrybucji, co gwarantuje pełną gotowość do pracy bezpośrednio po uruchomieniu pliku.

## 2.5 Ograniczenia projektowe

Warstwa wizualna aplikacji została zrealizowana przy użyciu frameworka Kivy, co pozwoliło na sprawne wdrożenie wieloplatformowego interfejsu użytkownika. Obecna wersja aplikacji jest w pełni zoptymalizowana pod kątem środowiska Windows oraz Linux. W przypadku uruchomienia aplikacji w systemie macOS, mogą występować marginalne różnice w skalowaniu wybranych komponentów wizualnych wynikające z odmiennych mechanizmów renderowania czcionek oraz gestów w tym środowisku. Rozbieżności te mają charakter wyłącznie kosmetyczny i nie wpływają na funkcjonalność ani stabilność działania systemu.

Zespół projektowy posiada świadomość powyższej charakterystyki i planuje w kolejnych fazach rozwoju produktu wdrożenie dedykowanej kompilacji, która zapewni pełną spójność wizualną z wytycznymi projektowymi systemu macOS.

## 2.6 Dokumentacja użytkownika

- Nawigacja: Moduły systemu dostępne są poprzez panel nawigacyjny (lewa strona okna). Zmiana języka oraz powiadomienia są obsługiwane z poziomu prawego górnego rogu.
- Kalendarz: Nawigacja czasowa odbywa się za pomocą dedykowanych przycisków. Wybór kafelka przedmiotu umożliwia zarządzanie notatką.
- Dodawanie przedmiotów/wydarzeń: Proces opiera się na wypełnieniu formularzy i zatwierdzeniu wprowadzonych danych.
- Zarządzanie: Edycja i usuwanie wpisów odbywa się poprzez interakcję z kaflami (z użyciem przycisków funkcyjnych).
- Progress Tracker: Modyfikacja punktacji odbywa się przyrostowo/dekrementacyjnie przy użyciu przycisków „+” oraz „-”.

## 2.7 Założenia i zależności

Aplikacja Student Planner nie wymaga do poprawnego funkcjonowania zewnętrznych systemów informatycznych.

# 3. Interfejsy zewnętrzne

## 3.1 Interfejs użytkownika (UI)

Interfejs użytkownika został zaprojektowany z myślą o maksymalnej czytelności i ergonomii pracy, opierając się na przejrzystym układzie panelowym, który segreguje funkcjonalności systemu w logiczne bloki:

- Panel nawigacyjny (sekcja boczna): Stały element interfejsu zapewniający szybki dostęp do kluczowych modułów aplikacji, takich jak Kalendarz, Rejestr Przedmiotów, Moduł Wydarzeń, Repozytorium Zasad oraz Tracker Postępów. W dolnej części panelu zintegrowano licznik odliczający czas pozostały do sesji egzaminacyjnej, co pozwala na stałe monitorowanie zbliżających się terminów.
- Obszar centralny (obszar roboczy): Dynamiczna przestrzeń przeznaczona do prezentacji aktualnie wybranego modułu. Zastosowano tu elastyczny system widoków, który zapewnia czytelność niezależnie od ilości wyświetlanych danych.
- Komponenty funkcyjne: Zarządzanie danymi odbywa się poprzez dedykowane formularze wejściowe, czytelne listy rekordów oraz okna modalne (*pop-ups*). Rozwiązanie to pozwala na efektywną interakcję z systemem bez konieczności przeładowywania głównego widoku, co znacząco poprawia komfort codziennego użytkowania.

### 3.2 Interfejsy sprzętowe

Aplikacja została zoptymalizowana pod kątem kompatybilności z szerokim spektrum urządzeń klasy PC oraz laptopów, spełniających minimalne wymagania systemowe. Interfejs systemu został zaprojektowany w sposób skalowalny, co zapewnia poprawne wyświetlanie komponentów na monitorach o różnej rozdzielczości i proporcjach ekranu. Dzięki niskim wymaganiom w zakresie zasobów systemowych, Student Planner działa stabilnie zarówno na starszych jednostkach, jak i nowoczesnych stacjach roboczych, zapewniając przy tym płynną reakcję na standardowe urządzenia wskazujące, takie jak mysz czy gładzik.

## 4. Funkcje systemowe

Wykaz funkcjonalności:

- Zarządzanie przedmiotami (dodawanie, edycja).
- Obsługa kalendarza (podgląd tygodniowy, notatki, wydarzenia).
- Moduł odliczania czasu do sesji.
- Konfiguracja języka.

- Moduł powiadomień.
- Zarządzanie wydarzeniami (dodawanie, usuwanie, modyfikacja).
- Monitorowanie postępów (Progress Tracker).
- Repozytorium zasad zaliczenia.

## 5. Wymagania niefunkcjonalne

### 5.1 Wydajność

System został zaprojektowany w oparciu o architekturę *offline-first*, co eliminuje opóźnienia wynikające z komunikacji sieciowej. Aplikacja charakteryzuje się minimalnym zapotrzebowaniem na zasoby procesora i pamięci operacyjnej, co zapewnia wysoką responsywność interfejsu (czas reakcji na zdarzenia wejściowe poniżej progu odczuwalnego przez użytkownika) nawet na urządzeniach o ograniczonej wydajności.

### 5.2 Bezpieczeństwo

Bezpieczeństwo danych opiera się na lokalnej izolacji. Ze względu na brak zewnętrznych połączeń sieciowych, ryzyko nieautoryzowanego dostępu do informacji z zewnątrz zostało zminimalizowane. integralność danych zapewniana jest przez transakcyjność operacji w bazie danych (wykorzystanie mechanizmu *atomic commits*), co chroni rejestry przed uszkodzeniem w przypadku nagłego przerwania pracy aplikacji.

### 5.3 Atrybuty jakościowe systemu

- **Użyteczność:** Projekt interfejsu opiera się na zasadach ergonomii poznawczej, co gwarantuje intuicyjną nawigację oraz szybką naukę obsługi systemu przez nowego użytkownika.
- **Niezawodność:** Kluczowym wymogiem jest bezbłądność operacji CRUD (Create, Read, Update, Delete). Każda zmiana w bazie danych podlega weryfikacji, co zapewnia pełną spójność przechowywanych informacji.
- **Przejrzystość:** Logika prezentacji danych została zoptymalizowana pod kątem maksymalizacji czytelności, przy zachowaniu estetycznej i spójnej szaty graficznej.

### 5.4 Model ról użytkownika

System został zaprojektowany w oparciu o model użytkownika typu *Single-User*, co oznacza, że osoba korzystająca z aplikacji posiada pełny, nieograniczony dostęp do wszystkich modułów oraz danych zgromadzonych w lokalnej bazie. Taka architektura upraszcza zarządzanie dostępem i zapewnia użytkownikowi pełną suwerenność nad zgromadzonymi informacjami akademickimi.

## **6. Pozostałe wymagania**

- Dokładny opis wymagań biznesowych, wymagań użytkownika oraz wymagań funkcjonalnych dostępny w dokumencie z dokumentacją analityczną projektu

## **7. Załączniki**

- Diagramy (czynności, przypadków użycia, klas).
- Opis klas.
- Dokumentacja testów automatycznych.
- Wykres Gantta pierwotny oraz faktyczny.
- Dokumentacja analityczna projektu.